```
Start coding or generate with AI.
```

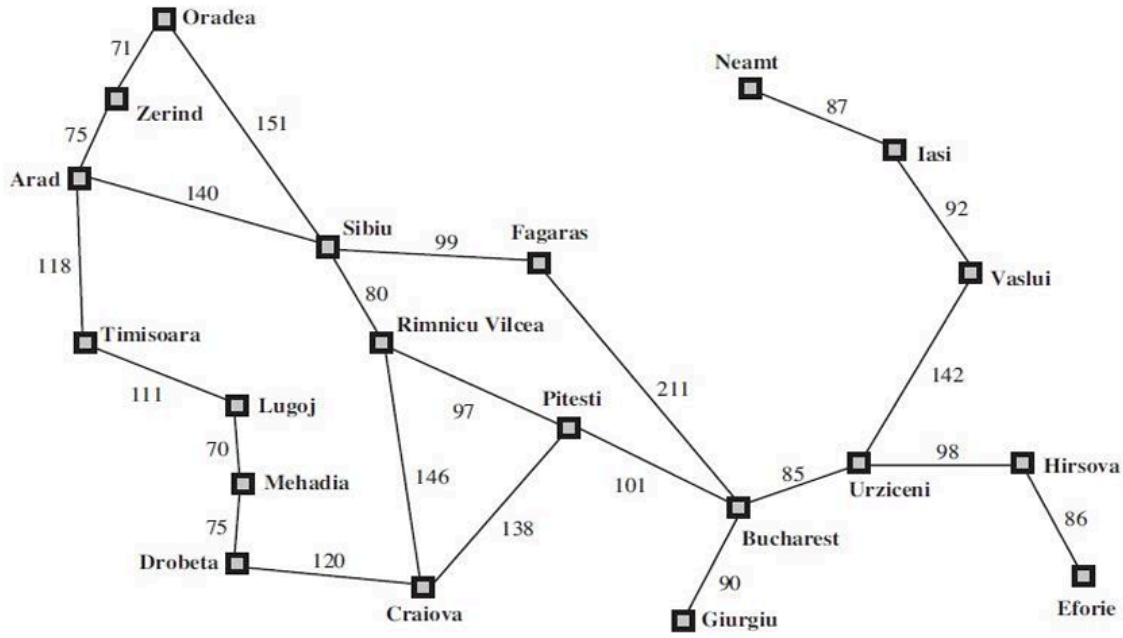

Figure: Graph for Travelling Salesman Problem

STEP:1 CREATE THE GRAPH

```
graph = {
    'Arad': [('Zerind', 75), ('Sibiu', 140), ('Timisoara', 118)],
    'Zerind': [('Oradea', 71)],
    'Oradea': [('Sibiu', 151)],
    'Timisoara': [('Lugoj', 111)],
    'Lugoj': [('Mehadia', 70)],
    'Mehadia': [('Drobeta', 75)],
    'Drobeta': [('Craiova', 120)],
    'Craiova': [('Pitesti', 138)],
    'Sibiu': [('Fagaras', 99), ('Rimnicu Vilcea', 80)],
    'Rimnicu Vilcea': [('Pitesti', 97)],
    'Fagaras': [('Bucharest', 211)],
    'Pitesti': [('Bucharest', 101)],
    'Bucharest': []
}
```

✓ Step 2: DFS Code (Path + Distance)

```
def dfs(graph, start, goal, visited=None, path=None, cost=0):
    if visited is None:
        visited = set()
    if path is None:
```

```
    if path is None:
        path = []

    visited.add(start)
    path.append(start)

    if start == goal:
        return path, cost

    for neighbour, distance in graph[start]:
        if neighbour not in visited:
            result = dfs(graph, neighbour, goal, visited, path, cost + distance)
            if result is not None:
                return result

    path.pop()
    return None


# Driver Code
start = 'Arad'
goal = 'Bucharest'

result = dfs(graph, start, goal)

if result:
    path, distance = result
    print("Path:", " -> ".join(path))
    print("Total Distance:", distance)
else:
    print("No path found")
```

```
Path: Arad -> Zerind -> Oradea -> Sibiu -> Fagaras -> Bucharest
Total Distance: 607
```